

Here's the first test of the function `get_formatted_name()`:

```
test_name
_function.py  from name_function import get_formatted_name

❶ def test_first_last_name():
    """Do names like 'Janis Joplin' work?"""
❷     formatted_name = get_formatted_name('janis', 'joplin')
❸     assert formatted_name == 'Janis Joplin'
```

Before we run the test, let's take a closer look at this function. The name of a test file is important; it must start with `test_`. When we ask pytest to run the tests we've written, it will look for any file that begins with `test_`, and run all of the tests it finds in that file.

In the test file, we first import the function that we want to test: `get_formatted_name()`. Then we define a test function: in this case, `test_first_last_name()` ❶. This is a longer function name than we've been using, for a good reason. First, test functions need to start with the word *test*, followed by an underscore. Any function that starts with `test_` will be *discovered* by pytest, and will be run as part of the testing process.

Also, test names should be longer and more descriptive than a typical function name. You'll never call the function yourself; pytest will find the function and run it for you. Test function names should be long enough that if you see the function name in a test report, you'll have a good sense of what behavior was being tested.

Next, we call the function we're testing ❷. Here we call `get_formatted_name()` with the arguments 'janis' and 'joplin', just like we used when we ran *names.py*. We assign the return value of this function to `formatted_name`.

Finally, we make an assertion ❸. An *assertion* is a claim about a condition. Here we're claiming that the value of `formatted_name` should be 'Janis Joplin'.

Running a Test

If you run the file *test_name_function.py* directly, you won't get any output because we never called the test function. Instead, we'll have pytest run the test file for us.

To do this, open a terminal window and navigate to the folder that contains the test file. If you're using VS Code, you can open the folder containing the test file and use the terminal that's embedded in the editor window. In the terminal window, enter the command **pytest**. Here's what you should see:

```
$ pytest
===== test session starts =====
❶ platform darwin -- Python 3.x.x, pytest-7.x.x, pluggy-1.x.x
❷ rootdir: /.../python_work/chapter_11
❸ collected 1 item

❹ test_name_function.py . [100%]
===== 1 passed in 0.00s =====
```
